

Running SD

The service, starting and stopping, and what to do when the last shutdown was not a clean one.

SD is a Windows service and it is already running. Nobody types `sd -start` any more.

Display name	String Database (SD)
Service name	<code>SD</code>
Start type	automatic — Windows starts it at every boot
Created by	the installer, which also starts it
Removed by	the uninstaller

This is a change from the Linux original, where `sd -start` had to be typed after every restart.

Starting and stopping

Stopping the service stops SD and ends every session on the machine, which is exactly what `sd -stop` has always done. Either is fine:

```
Stop-Service SD
Start-Service SD
```

```
sd -stop
sd -start
```

`sd -start` and `sd -stop` tell the truth now

Before this port, both answered from the shared memory segment, which outlives the daemon — so both could report success while doing nothing. Four things changed, and each is a state you may hit while testing.

"SD is already started" IS NOW ONLY SAID WHEN THE DAEMON REALLY IS RUNNING, and it tells you the process id — the Windows one, the number Task Manager and `Stop-Process` use.

IF THE SEGMENT IS THERE BUT THE DAEMON IS NOT — what a killed or crashed SD leaves behind — `sd -start` says so and tells you to run `sd -stop` first. It used to say *"SD is already started"* and do nothing, **leaving the system unusable while the command that would fix it reported success.**

It does **not** clear the wreckage for you: that would end any sessions still attached to the segment. **The count of those is printed so you can decide.**

`sd -stop` **now checks that the daemon actually stopped.** A daemon started from an elevated session cannot be stopped from an ordinary one — Windows refuses the signal — and that used to be silent, leaving a daemon running against a segment nothing else could see. You now get a warning naming the process id and the command to stop it with.

Known limit. If the segment has already gone, `sd -stop` has nowhere left to read the daemon's process id from and cannot report on it at all. Check by hand:

```
Get-Process sdwind
```

After an unclean shutdown

If SD is stopped abruptly — the power goes, or the process is killed — it leaves a shared memory segment behind. **On Windows that survives a reboot, where on Linux it would not.**

Earlier builds of this port refused to start on the next boot and said *"Run sd -stop to clear it"*, so **the machine came up with SD unavailable to everybody** until somebody logged in and typed it by hand. There was no Linux behaviour to inherit here: on Linux the segment does not survive the reboot at all.

Nothing from before a restart can still be using that segment, so SD now discards it and starts normally, printing:

```
Discarding the shared segment left by the previous boot -  
SD did not shut down cleanly.
```

This changes nothing while the machine is running. A segment belonging to a live SD is still never touched, and `sd -start` still refuses to disturb a system that is already up.

SD will not start a second time inside itself

If you leave SD with `sh` and then type `sd` in that shell, it says so and returns you to the session you already have.

Worth knowing alongside it: `sh` **itself needs either an elevated session or a `yes` in `os.users`**, so an ordinary account cannot leave SD this way at all, and one reached over ssh never can. See Administrator commands.

The command line

```
sd                enter the SD account named after your Windows login
sd -a             prompt for an account
sd -a<name>       enter account <name> -- refused unless it is your own
sd <command>      run one command -- needs elevation, or batch.jobs
sd -quiet         suppress the displays on entry
sd -u             list current users
sd -k <n> | -k all log out user n, or everybody
sd -start         start the system
sd -stop          stop the system
sd --version      report the version
sd --help         this summary
```

Three of these behave differently from what you may expect.

`sd -a<name>` **is refused unless `<name>` is your own account**. An administrator no longer opens somebody else's account without ever being in their own — they arrive in their own and reach the rest with `logto`, which is where SD checks whether they are allowed in.

`sd <command>` **needs an elevated session**, or an entry for that account in `batch.jobs`. **Any account can be given one, whatever its tier** — standard, programmer or administrator. The account type does not decide what may run; the administrator's list does. That is what makes scheduled jobs possible without handing them administrator rights — see Scheduled jobs.

`sd <command>` **runs and exits, and is never asked to set a password**. Nor is any session with no terminal — a scheduled task, or a piped script. **Only an interactive `sd` with no command after it still asks**, and then only of an administrator account that has no password yet.

Earlier builds of this port reached the "*needs a password*" prompt and blocked for ever on a read that never got input, with nothing in any log because nothing had gone wrong from SD's side.

Where things are

Binaries	<code>C:\Program Files\SD\usr\bin\</code>
----------	---

The changelog	<code>C:\Program Files\SD\changelog</code>
---------------	--

Configuration	C:\ProgramData\SD\sd.conf
The database	C:\ProgramData\SD\sdsys\
Accounts	C:\ProgramData\SD\user_accounts\, ... \group_accounts\
Audit trail	C:\ProgramData\SD\sdsys\audit
Error log	C:\ProgramData\SD\sdsys\errlog
Elevation helper log	C:\ProgramData\SD\sd-elevate.log

Do not move the binaries. `usr\bin` is load-bearing: shipping `msys-2.0.dll` beside the executable relocates the POSIX root to the DLL's directory minus two components, and only that depth puts `/` on `C:\Program Files\SD\`.

The DLLs ship beside `sd.exe` deliberately — Windows searches the executable's own directory before `PATH`, which avoids Git for Windows's rival `msys-2.0.dll` being picked up. **That failure makes SD report "SD has not been started" while it is running**, which is worth recognising because it looks like nothing else.

The Start Menu

SD	starts <code>sd.exe</code> in the data directory
Check the SD installation	the post-install check, re-runnable at any time. Closes on a keypress
